

Lab 03 — Praktijklab: leven, signalen en dood van een container

Doel: elk gedrag uit de cursus zelf uitlokken — de onmiddellijke stop, de 10 seconden doodsstrijd, code 137, de automatische herstart — en zien wie, zonder daemon, over je containers waakt.

Vereisten — Labs 01 en 02 afgewerkt. Images `alpine` en `nginx:alpine` aanwezig.

Geleverde bestanden — `files/demarrage-casse.sh` (kapotte opstart) en `files/demarrage-correct.sh` (correcte opstart), gebruikt in stap 3.

Stap 1 — De toestanden, één voor één

```
podman create --name toestand alpine sleep 120
podman ps -a --filter name=toestand --format 'table {{.Names}}\t{{.Status}}'
```

Observeer de status `Created`: de container bestaat, er draait geen enkel proces.

```
podman start toestand
podman ps --filter name=toestand --format '{{.Status}}'
podman pause toestand && podman ps -a --filter name=toestand --format '{{.Status}}'
podman unpause toestand && podman ps --filter name=toestand --format '{{.Status}}'
podman stop -t 2 toestand && podman ps -a --filter name=toestand --format '{{.Status}}'
podman start toestand && podman ps --filter name=toestand --format '{{.Status}}'
podman rm -f -t 0 toestand
```

Observeer de opeenvolging `Up 1 second`, `Paused`, `Up 5 seconds`, een waarschuwing `StopSignal SIGTERM failed to stop container toestand in 2 seconds, resorting to SIGKILL`, `Exited (137)`, en dan opnieuw `Up`.

Uitleg. Een `Exited`-container is herstartbaar: hij heeft zijn configuratie en zijn schrijflaag behouden. Merk op dat `podman ps` zonder `-a` een gepauzeerde container **niet toont**: hij is niet "running".

Stap 2 — Waarom een container vanzelf stopt

```
podman run --name poging1 alpine
podman ps -a --filter name=poging1 --format '{{.Status}}'
```

Observeer `Exited (0)`: het standaardcommando van `alpine` is `/bin/sh`, dat zonder invoer meteen afsluit.

```
podman run -d --name poging2 nginx:alpine
podman ps --filter name=poging2 --format '{{.Status}}'
```

Observeer `Up`: nginx draait op de voorgrond.

```
podman run --rm alpine sh -c 'sleep 60 & echo "op de achtergrond gestart"'
```

Observeer dat het commando **onmiddellijk** terugkomt, hoewel er wel degelijk een `sleep 60` gestart is.

Uitleg. De `&` heeft `sleep` losgekoppeld; de shell voerde `echo` uit en sloot af. PID 1 dood, dus de container vernietigd, `sleep` erbij. Dat is **dé** valkuil nummer één van opstartscripts.

Kijk wie over `poging2` waakt terwijl hij draait:

```
podman inspect --format '{{.State.CommonPid}}' poging2
ps -o pid,ppid,user,comm -p $(podman inspect --format '{{.State.CommonPid}}' poging2)
```

Observeer een `conmon`-proces, onder **jouw** gebruiker: het is de toezichhouder die Podman achter elke container laat — de enige "daemon" die je nog hebt, en hij weegt maar enkele honderden KB.

```
podman rm poging1 ; podman rm -f -t 0 poging2
```

Stap 3 — Het kapotte opstartscript, en de correctie

Kopieer de twee geleverde scripts:

```
mkdir -p ~/labo-docker/03 && cd ~/labo-docker/03
cp <pad-van-het-lab>/files/*.sh . && chmod +x *.sh
cat demarrage-casse.sh demarrage-correct.sh
```

Voer het eerste uit **in** een container, met de huidige map gemount:

```
podman run --rm -v "$PWD":/scripts alpine /scripts/demarrage-casse.sh
```

Observeer de getoonde boodschap, en dan meteen de prompt terug: de container is al dood en verwijderd.

```
podman run -d --name correct -v "$PWD":/scripts alpine /scripts/demarrage-correct.sh
podman ps --filter name=correct --format '{{.Status}}'
podman top correct
```

Observeer dat de container `Up` blijft, en dat `podman top` `sleep 300` als PID 1 toont — en **geen** ouderproces `sh`.

Uitleg. De `exec` van het tweede script heeft de shell **vervangen** door het eindcommando, dat PID 1 erft. Dat is precies wat de `exec`-vorm van een `ENTRYPOINT` doet, gezien in lab 04.

→ LINUX

`exec` is een ingebouwd shellcommando dat de gelijknamige systeemaanroep aanroept: het huidige proces laat zijn programma (de shell) vallen en laadt het gevraagde programma **op zijn plaats**, met behoud van zijn PID. Zonder `exec` maakt de shell een kind aan (`fork`) en wacht. Met `exec` is er helemaal geen shell meer.

```
podman rm -f -t 0 correct
```

Stap 4 — De 10 seconden doodsstrijd

Meet een stop op een proces dat `SIGTERM` negeert:

```
podman run -d --name wacht alpine sleep 300
time podman stop wacht
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' wacht
podman rm wacht
```

Observeer de waarschuwing `StopSignal SIGTERM failed to stop container wacht in 10 seconds, resorting to SIGKILL`, `real 0m10.1s` en `code=137 oom=false`.

Begin opnieuw met een mini-init:

```
podman run -d --init --name wacht alpine sleep 300
podman exec wacht ps -o pid,comm
time podman stop wacht
podman inspect --format 'code={{.State.ExitCode}}' wacht
podman rm wacht
```

Observeer `1 podman-init` en dan `sleep` als PID 2, een stop in `0m0.1s` en `code=143`.

En met een applicatie die haar signalen correct behandelt:

```
podman run -d --name web nginx:alpine
time podman stop web
podman inspect --format 'code={{.State.ExitCode}}' web
podman rm web
```

Observeer een onmiddellijke stop en `code=0`.

Uitleg. Drie gedragingen, drie oorzaken. `sleep` als PID 1 **negeert** `SIGTERM` (bescherming van de kernel): de engine wacht en doodt → `137`. Met `--init` is `sleep` geen PID 1 meer, de standaardactie geldt → `143`. `nginx` installeert een signaalhandler en stopt netjes (met code `0`, omdat `nginx` ervoor koos normaal af te sluiten). Die tien seconden, vermenigvuldigd met je containers, zijn de onverklaarde duur van je heruitrollen.

Je kunt de respijtperiode inkorten — zonder de oorzaak te verhelpen:

```
podman run -d --name wacht alpine sleep 300
time podman stop -t 2 wacht
podman rm wacht
```

Observeer `real 0m2.1s`, nog altijd met code `137`.

Stap 5 — Exitcodes lezen

```
podman run --rm alpine sh -c 'exit 0' ; echo "code=$?"
podman run --rm alpine sh -c 'exit 3' ; echo "code=$?"
podman run --rm alpine onbestaand-commando ; echo "code=$?"
```

Observeer `0`, `3`, en dan `127` met `Error: crun: executable file onbestaand-commando not found in $PATH`.

```
podman run -d --name gedood alpine sleep 300
podman kill gedood
podman inspect --format 'code={{.State.ExitCode}}' gedood
podman rm gedood
```

Observeer `137`, deze keer onmiddellijk: `kill` wacht niet.

Veroorzaak nu een echt geheugentekort:

```
podman run --name oom --memory=32m --memory-swap=32m alpine sh -c 'head -c 100m /dev/zero |
tail'
echo "code=$?"
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' oom
podman rm oom
```

Observeer `code=137` en deze keer `oom=true`: dezelfde code, een andere oorzaak, en alleen `inspect` maakt het verschil.

Uitleg. Boven 128 wijst de code op een dood door signaal: `code - 128` geeft het signaalnummer. `127` daarentegen is een opstartfout: de applicatie is nooit gestart.

Stap 6 — `exec` tegenover `attach`

```
podman run -d --name web nginx:alpine
podman exec web nginx -v
podman exec -it web sh
```

Typ in de verkregen shell:

```
ps -o pid,comm
exit
```

Observeer dat `nginx` PID 1 is, gevolgd door zijn *workers*, en dat je `sh` een andere PID heeft. Bij het verlaten van de shell is de container **nog altijd** `Up`.

```
podman ps --filter name=web --format '{{.Status}}'
```

Uitleg. `exec` heeft een **nieuw** proces aangemaakt in de namespaces van de container. Het verlaten heeft geen invloed op PID 1. `attach` daarentegen zou je aan nginx zelf koppelen: een `Ctrl+C` zou het stoppen. Om logs te lezen, gebruik altijd:

```
podman logs --tail 5 web
podman logs -f --since 1m web      # Ctrl+C stopt hier alleen de weergave
```

Stap 7 — Logs komen alleen van `stdout`

```
podman run --rm --name logs-demo alpine sh -c \
  'echo "ik ga naar stdout"; echo "ik ga naar een bestand" > /tmp/app.log; sleep 1'
```

Observeer dat alleen de eerste regel verschijnt.

```
podman run -d --name logs-demo alpine sh -c \
  'echo "zichtbaar"; echo "onzichtbaar" > /tmp/app.log; sleep 120'
podman logs logs-demo
podman exec logs-demo cat /tmp/app.log
podman rm -f -t 0 logs-demo
```

Observeer dat `podman logs` `zichtbaar` toont en dat de inhoud van het bestand alleen bereikbaar is door in de container te gaan.

Uitleg. `conmon` vangt alleen `stdout` en `stderr` van PID 1 op. Daarom moet een gecontaineriseerde applicatie naar de console loggen — en daarom mag je in een gecontaineriseerde Spring Boot geen `logging.file.name` configureren.

Stap 8 — Automatische herstart

```
podman run -d --restart=on-failure:3 --name onstabiel alpine \
  sh -c 'echo "start $(date +%T)"; sleep 3; exit 1'
sleep 20
podman ps -a --filter name=onstabiel --format '{{.Names}} {{.Status}}'
podman inspect --format 'herstarts={{.RestartCount}} code={{.State.ExitCode}}' onstabiel
podman logs onstabiel
```

Observeer een `RestartCount` van `3`, een status `Exited (1)`, en **vier** regels "start" in de logs: de eerste poging plus drie hervattingen.

Uitleg. Logs stapelen zich op van de ene uitvoering naar de andere op dezelfde container: de eerste regel is de oorspronkelijke oorzaak. `.State` daarentegen beschrijft alleen de **laatste** uitvoering.

```
podman rm onstabiel
podman events --since 2m --until 1s | grep onstabiel | awk '{print $5, $6}' | uniq -c
```

Observeer het logboek van gebeurtenissen: `container start`, `container died`, `container restart` ... Het is de enige plek waar je de *geschiedenis* van een container ziet, niet alleen zijn toestand.

Ga ten slotte de in de cursus aangekondigde onverenigbaarheid na:

```
podman run --rm -d --restart=always nginx:alpine
```

Observeer `Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"`.

PODMAN

En na een reboot? Test het: `podman run -d --restart=always --name overlever nginx:alpine`, sluit dan **al** je Ubuntu-vensters en doe vanuit PowerShell `ws -shutdown`. Open Ubuntu opnieuw: `podman ps` is leeg. Niemand heeft het beleid herlezen — er is geen daemon. Op een server is die rol voor `systemd` via een Quadlet-bestand (lab 10). Op je werkpost is dit aanvaardbaar gedrag: je ontwikkelcontainers hoeven geen herstart te overleven. Daarna `podman rm -f -t 0 overlever`.

Stap 9 — Bewijs uit een dode container halen

```
podman run --name autopsie alpine sh -c 'echo "belangrijk spoor" > /rapport.txt; exit 2'
podman ps -a --filter name=autopsie --format '{{.Status}}'
podman cp autopsie:/rapport.txt ./rapport.txt
cat rapport.txt
```

Observeer dat het bestand te recupereren is terwijl de container `Exited (2)` is.

```
podman rm autopsie
podman cp autopsie:/rapport.txt ./ander.txt
```

Observeer `Error: container "autopsie" does not exist`: eens de container verwijderd, is alles verloren.

Uitleg. Exploitatieregel: **eerst inspecteren, dan verwijderen**. `cp`, `logs` en `inspect` werken op een gestopte container, nooit op een verwijderde.

Opruimen

```
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
podman rm -f -t 0 wacht web correct onstabiel autopsie poging1 poging2 gedood toestand oom
overlever 2>/dev/null
rm -f ~/labo-docker/03/rapport.txt
podman ps -a --format '{{.Names}}'
```

Observeer dat geen enkele container van dit lab overblijft. De images `alpine` en `nginx:alpine` blijven bewaard.

Wat je nu moet kunnen beweren

- Een container sterft met zijn PID 1 — je hebt de drie gevallen uitgelokt.
- `sleep` als PID 1 negeert `SIGTERM`; `--init` verhelpt het symptoom, `exec` de oorzaak.
- `137` = gedood (door `stop`, `kill` of de OOM killer — `inspect` beslist), `143` = netjes gestopt, `127` = commando niet gevonden.
- `exec` maakt een proces aan, `attach` koppelt aan PID 1.
- `podman logs` toont alleen `stdout` / `stderr`, opgevangen door `conmon`.
- `podman rm` vernietigt logs en bewijzen: eerst inspecteren.
- Zonder daemon overleeft een *restart policy* geen `ws -shutdown`.